Quiz 6 - Introduction to Functions in Python

Ungraded in-class practice. Exam-style: code snippet + 5 true/false statements.

- Review: one question at a time
- Scope: full DataCamp course (Ch 1-3)
- Plan: 12 questions (Q11-Q12 are tricky review)

Question 1 (Chapter 1 - Defining a function, parameters, return, and docstrings)

Consider the following code:

```
def simple_return(start_price, end_price):
    """Return the simple price return."""
    return (end_price - start_price) / start_price

r = simple_return(100, 105)
print(r)
```

A. TRUE - r is equal to 0.05.
   Explanation: The price rises by 5 on a starting price of 100, so the simple `return` is 5/100 = 0.05.

B. TRUE - `start_price` and `end_price` are parameters of the function.
   Explanation: They appear in the function definition and receive argument values when the function is called.

C. TRUE - `return` sends the calculated value back to the caller.
   Explanation: The `return` statement makes the function produce a value that can be stored in r.

D. FALSE - The docstring changes the numerical `result` produced by the function.
   Explanation: The triple-quoted text documents the function; it does not alter the `return` calculation.

E. FALSE - Calling `simple_return`(100) would work because `end_price` would automatically become 0.
   Explanation: `end_price` has no default value, so omitting it raises a TypeError.

Question 2 (Chapter 1 - Multiple parameters and keyword arguments)

Consider the following code:

```
def operating_profit(revenue, cogs, opex):
    return revenue - cogs - opex

profit = operating_profit(1000, cogs=400, opex=250)
print(profit)
```

A. TRUE - profit is equal to 350.
   Explanation: The function calculates 1000 - 400 - 250 = 350.

B. TRUE - `cogs`=400 and `opex`=250 are keyword arguments.
   Explanation: Those arguments are supplied by explicitly naming the parameters they belong to.

C. TRUE - The keyword arguments could be written as `opex`=250, `cogs`=400 without changing the `result`.
   Explanation: Keyword arguments are matched by parameter name, so their written order can change.

D. FALSE - `cogs` and `opex` are default parameters because their values are supplied in the function call.
   Explanation: A default value must be defined in the function header, such as `cogs`=400; these parameters have no defaults.

E. FALSE - The call `operating_profit`(`revenue`=1000, 400, 250) is valid Python syntax.
   Explanation: Positional arguments cannot be placed after a keyword argument in this call.

Question 3 (Chapter 1 - Multiple return values and tuples)

Consider the following code:

```
def trade_stats(entry_price, exit_price):
    pnl = exit_price - entry_price
    trade_return = pnl / entry_price
    return pnl, trade_return

result = trade_stats(50, 55)
print(result)
```

A. TRUE - `result` is a tuple containing two values.
Explanation: Returning two comma-separated values packages them into a tuple.

B. TRUE - `result`[0] is equal to 5.
Explanation: The first `return`ed value is `pnl`, calculated as 55 - 50 = 5.

C. TRUE - `result`[1] is equal to 0.10.
Explanation: The trade `return` is 5/50 = 0.10.

D. TRUE - The assignment `pnl`, `ret` = `trade_stats`(50, 55) would be valid.
Explanation: The two values in the `return`ed tuple can be unpacked into two variables.

E. FALSE - `result` is a list because the function `return`s more than one value.
Explanation: Comma-separated `return` values are `return`ed as a tuple, not a list.

Question 4 (Chapter 2 - Default arguments)

Consider the following code:

```
def after_tax_profit(ebit, tax_rate=0.25):
    return ebit * (1 - tax_rate)

base = after_tax_profit(200)
low_tax = after_tax_profit(200, 0.20)
print(base, low_tax)
```

A. TRUE - base is equal to 150.0.
Explanation: Because `tax_rate` is omitted, the default 0.25 is used: 200 x 0.75 = 150.0.

B. TRUE - low_tax is equal to 160.0.
Explanation: The supplied tax rate overrides the default: 200 x 0.80 = 160.0.

C. TRUE - The default value 0.25 is used only when `tax_rate` is not supplied.
Explanation: A caller-provided argument replaces the default value for that call.

D. FALSE - Calling `after_tax_profit`() with no arguments would work because both parameters have defaults.
Explanation: ebit has no default value and is required.

E. TRUE - `after_tax_profit`(200, `tax_rate`=0.30) would `return` 140.0.
Explanation: The calculation is 200 x (1 - 0.30) = 140.0.

Question 5 (Chapter 2 - Global and local scope)

Consider the following code:

```
risk_limit = 1000000

def set_limit():
    global risk_limit
    risk_limit = 750000

set_limit()
print(risk_limit)
```

A. TRUE - After set_limit() runs, `risk_limit` is equal to 750000.
Explanation: The function assigns 750000 to the `global` variable.

B. TRUE - The `global` keyword makes the assignment refer to the `risk_limit` defined outside the function.
Explanation: `global` tells Python that assignments to `risk_limit` should target the `global` scope.

C. TRUE - If the `global` line were removed, assigning `risk_limit` = 750000 inside the function would leave the outside `risk_limit` equal to 1000000.
Explanation: Without `global`, that assignment would create a local variable instead of changing the `global` one.

D. FALSE - A function must use `global` whenever it only reads a variable from `global` scope.
Explanation: A function can normally read a `global` variable without declaring it `global`; `global` is needed here because the function assigns to it.

E. FALSE - Declaring `risk_limit` as `global` makes the variable read-only inside the function.
Explanation: The purpose of `global` here is precisely to allow the function assignment to modify the `global` name.

Question 6 (Chapter 2 - Nested functions, nonlocal, and closures)

Consider the following code:

```
def make_budget_tracker(start_budget):
    budget = start_budget

    def spend(amount):
        nonlocal budget
        budget -= amount
        return budget

    return spend

tracker = make_budget_tracker(1000)
a = tracker(200)
b = tracker(150)
print(a, b)
```

A. TRUE - a is equal to 800.
Explanation: The first call subtracts 200 from the starting `budget` of 1000.

B. TRUE - b is equal to 650.
Explanation: The second call continues from 800 and subtracts another 150.

C. TRUE - `nonlocal budget` refers to `budget` in the enclosing `make_budget_tracker`() function.
Explanation: `nonlocal` targets a variable in an enclosing function scope rather than creating a new local variable.

D. TRUE - `tracker` can remember the updated `budget` even after `make_budget_tracker`() has finished running.
Explanation: The `return`ed nested function forms a closure that `ret`ains access to its enclosing state.

E. FALSE - Removing `nonlocal` while leaving `budget` -= amount unchanged would produce the same behavior.
Explanation: Without `nonlocal`, Python would treat `budget` as local to spend(), causing an UnboundLocalError when it is read before local assignment.

Question 7 (Chapter 2 - Variable-length positional arguments (*args))

Consider the following code:

```
def total_pnl(*pnls):
    return sum(pnls)

day_pnl = total_pnl(120, -40, 70)
print(day_pnl)
```

A. TRUE - Inside the function, `pnls` is a tuple.
Explanation: *args-style parameters collect extra positional arguments into a tuple.

B. TRUE - day_`pnl` is equal to 150.
Explanation: The function `sum`s 120 - 40 + 70 = 150.

C. TRUE - Calling `total_pnl`() with no arguments would `return` 0.
Explanation: With no positional arguments `pnls` is an empty tuple, and `sum`(()) `return`s 0.

D. TRUE - `sum`() is available here from Python built-in scope, so no import is required.
Explanation: `sum` is one of Python's built-in functions.

E. FALSE - Calling `total_pnl`([120, -40, 70]) would `return` 150.
Explanation: That passes one list as a single argument; `sum` then tries to add the list to its numeric starting value and raises a TypeError.

Question 8 (Chapter 2 - Variable-length keyword arguments (**kwargs))

Consider the following code:

```
def report_metrics(**metrics):
    return metrics

report = report_metrics(revenue=500, ebitda=120, debt=200)
print(report)
```

A. TRUE - `report` is a dictionary.
Explanation: **kwargs-style parameters collect keyword arguments into a dictionary.

B. TRUE - `report`["ebitda"] is equal to 120.
Explanation: The keyword name becomes a dictionary key whose value is 120.

C. TRUE - **`metrics` allows the function to accept a variable number of keyword arguments.
Explanation: The caller can provide zero or more named arguments and they are collected in `metrics`.

D. FALSE - Calling `report_metrics`(500, 120, 200) would create the same dictionary.
Explanation: The function accepts variable keyword arguments, not unnamed positional arguments.

E. FALSE - Every possible key passed to `report_metrics`() must be declared in advance in the function header.
Explanation: **kwargs is specifically designed to collect keyword names that are not individually listed as parameters.

Question 9 (Chapter 3 - Lambda functions and map())

Consider the following code:

```
prices = [100, 105, 110]
adjusted = list(map(lambda p: p - 1, prices))
print(adjusted)
```

A. TRUE - `adjusted` is equal to [99, 104, 109].
Explanation: The `lambda` subtracts 1 from each price.

B. TRUE - `lambda` p: p - 1 defines a small anonymous function.
Explanation: `lambda` creates a function expression without giving it a normal def-style name.

C. TRUE - `map()` applies the `lambda` to each element of `prices`.
Explanation: Each item from `prices` is passed through the function supplied to map.

D. FALSE - In Python 3, `map()` itself directly `returns` a list.
Explanation: map `returns` a map iterator; list(...) is used here to materialize the `results` as a list.

E. FALSE - The original `prices` list is changed to [99, 104, 109].
Explanation: map builds transformed `results` without modifying the original list in place.

Question 10 (Chapter 3 - Lambda functions and filter())

Consider the following code:

```
margins = [0.08, 0.15, 0.22, 0.05]
strong = list(filter(lambda m: m >= 0.15, margins))
print(strong)
```

A. TRUE - `strong` is equal to [0.15, 0.22].
Explanation: Only the `margins` meeting the m >= 0.15 condition are `ret`ained.

B. TRUE - The `lambda` `returns` a Boolean value for each margin.
Explanation: The comparison m >= 0.15 evaluates to either True or False.

C. TRUE - `filter()` keeps elements for which its function produces a truthy `result`.
Explanation: Values that satisfy the predicate are included in the filtered output.

D. TRUE - The original `margins` list remains unchanged.
Explanation: filter creates a filtered `result` rather than deleting elements from the source list.

E. FALSE - 0.08 appears in `strong` because it is a positive number.
Explanation: The filter condition is m >= 0.15, and 0.08 does not satisfy it.

Question 11 (Review - reduce() and cumulative calculations)

Consider the following code:

```
from functools import reduce

gross_factors = [1.02, 0.98, 1.05]
growth = reduce(lambda x, y: x * y, gross_factors)
print(growth)
```

A. TRUE - reduce must be imported from `functools` in this example.
Explanation: reduce is provided by the `functools` module rather than as a Python built-in.

B. TRUE - `growth` is approximately equal to 1.04958.
Explanation: The cumulative multiplication is 1.02 x 0.98 x 1.05 = 1.04958, apart from normal floating-point representation.

C. TRUE - `reduce()` repeatedly combines values until one accumulated `result` remains.
Explanation: The `lambda` first combines two values and then continues combining the accumulated `result` with the next value.

D. FALSE - Replacing reduce(...) with `sum`(`gross_factors`) would produce the same `result`.
Explanation: `sum` adds the factors to get 3.05, whereas reduce here multiplies them to obtain about 1.04958.

E. TRUE - reduce(`lambda` x, y: x * y, [1.02]) `returns` 1.02.
Explanation: With a one-element iterable and no initializer, that single element is the `result`ing accumulated value.

Question 12 (Review - try-except and raising errors)

Consider the following code:

```
def debt_ratio(debt, assets):
    if assets <= 0:
        raise ValueError("assets must be positive")
    return debt / assets

try:
    ratio = debt_ratio(300, 0)
except ValueError:
    ratio = None

print(ratio)
```

A. TRUE - `debt_ratio`(300, 0) raises a `ValueError`.
Explanation: assets is 0, so the condition assets <= 0 is met and the raise statement executes.

B. TRUE - The except block catches that `ValueError` and sets ratio to `None`.
Explanation: The raised exception matches except `ValueError`, so execution moves into that handler.

C. FALSE - After raise `ValueError`(...) executes, the function continues to `return` debt / assets.
Explanation: Raising the exception immediately interrupts normal execution of the function unless the exception is handled elsewhere.

D. TRUE - `debt_ratio`(300, 600) would `return` 0.5.
Explanation: Because assets is positive, no error is raised and 300/600 = 0.5.

E. FALSE - Without the `try-except` block, `debt_ratio`(300, 0) would quietly `return` `None`.
Explanation: Without a matching handler, the `ValueError` would propagate and interrupt normal program flow rather than automatically `return`ing `None`.